API Design

Overview

This document defines the API design for the Animal Adoption Management System (PetPals), a multi-shelter pet adoption platform. It covers the conventions, structure, and endpoint catalogue that all backend development will follow across every sprint.

The API is designed prior to implementation to ensure consistency across all modules and all six user roles: Admin, Shelter Staff, Adopter, Vet, Volunteer, and Donor. It follows REST principles and serves as the contract between the React frontend and the Node.js/Express backend.

This document covers naming conventions, versioning strategy, standard response and error structures, authentication and authorization overview, and the full resource endpoint catalogue. Implementation details such as database queries, middleware logic, and business rules are out of scope and live in the codebase.

1. Base URL Versioning

- All API requests are served from the following base URL:
  `https://petpals-api.up.railway.app/api/v1`

- During local development, the base URL is:
  `http://localhost:5000/api/v1`

- The version number is embedded directly in the URL path rather than passed as a request header. This makes the API version explicit and easy to test across tools like Postman and Swagger. If a breaking change is introduced in the future, a `/v2/` path will be added without deprecating `/v1/`, ensuring backward compatibility.
  A full endpoint URL follows this pattern:
  `GET https://petpals-api.up.railway.app/api/v1/pets`

2.Naming Conventions

No	Rule	Example
01	Use plural nouns for all resource names	`/pets`, `/adopters`, `/shelters`
02	Use kebab-case for multi-word resource names	`/adoption-applications`, `/pet-photos`
03	Use HTTP methods to convey actions (no verbs in URLs)	`GET /pets` not `/getPets`
04	Nest related resources one level deep to show ownership	`/shelters/:id/staff`, `/pets/:id/photos`

No	Rule	Example
05	Use `:id` path parameters to identify a specific record	`/pets/:id`, `/adopters/:id`
06	Use query parameters for filtering, sorting, and pagination	`/pets?species=dog&page=1&limit=20`

3.Standard Response Structure

Every API response follows a consistent JSON structure regardless of the endpoint or HTTP method. The success field is always present so the client can immediately determine whether the request succeeded without inspecting the HTTP status code.

3.1 Single Record

Returned when fetching or updating one specific resource.

```
{
  "success": true,
  "message": "Pet retrieved successfully",
  "data": {
    "petID": 1,
    "petName": "Buddy",
    "adoptionStatus": "available"
  }
}
```

3.2 List with Pagination

Returned when fetching multiple records.

```
{
  "success": true,
  "message": "Pets retrieved successfully",
  "data": [ ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 143,
    "totalPages": 8
  }
}
```

3.3 Error

Returned whenever a request fails. The `code` field is a machine-readable string the frontend can use to handle errors programmatically.

```
{
  "success": false,
  "message": "Pet not found",
  "error": {
    "code": "NOT_FOUND",
    "details": "No pet exists with ID 42"
  }
}
```

4. Error Codes

All API errors return a consistent error code in the response body alongside the appropriate HTTP status code.

Error Code	HTTP Status	When it is returned
`BAD_REQUEST`	400	Invalid or missing input in the request body
`UNAUTHORIZED`	401	No token provided or token is invalid/expired
`FORBIDDEN`	403	Token is valid but the role does not have permission
`NOT_FOUND`	404	The requested resource does not exist
`CONFLICT`	409	A duplicate record already exists (e.g. email already registered)
`VALIDATION_ERROR`	422	Request body failed validation rules
`INTERNAL_SERVER_ERROR`	500	Something unexpected went wrong on the server

Note the distinction between UNAUTHORIZED and FORBIDDEN: UNAUTHORIZED means the request lacks valid authentication (no token or expired token), while FORBIDDEN means the user is authenticated but does not have the required role to perform the action.

5. Authentication & Authorization

5.1 Authentication

The API uses JSON Web Tokens (JWT) for authentication and bcrypt for password hashing. The authentication flow is as follows:

1. The user submits their email and password to `POST /api/v1/auth/login`
2. The server verifies the password against the stored bcrypt hash
3. If valid, the server returns a short-lived access token (15 min) in the response body and sets a longer-lived refresh token (7 days) as an httpOnly cookie.

4. The client keeps the access token in memory and attaches it to every request as Authorization: Bearer. The refresh token is never touched by client code — the browser sends it automatically via the cookie.
5. When the access token expires, the client calls POST /auth/refresh-token (cookie- authenticated, no Bearer header) to get a new one, rather than requiring the user to log in again.

Tokens have an expiry time. If a token is expired or invalid, the server returns a `401 UNAUTHORIZED` error.

5.2 Authorization

Access control is enforced through Role-Based Access Control (RBAC). Each JWT payload contains the authenticated user's role, which Express middleware checks against the permissions required by the endpoint. The six roles in the system are:

Role	Description
Admin	Manages the shelter network and user accounts
Shelter Staff	Manages pets, applications, events, donors, and volunteers
Adopter	Browses pets, submits applications, and schedules appointments
Vet	Manages appointments and vaccination records
Volunteer	Views and updates assigned tasks and schedules
Donor	Makes and views donations

6.Endpoints by Domain

The following tables list all API endpoints grouped by resource domain. The Auth column indicates whether a valid JWT is required. The Roles column lists which user roles are permitted to access the endpoint.

6.1 Auth

Method	Endpoint	Description	Auth	Roles
POST	/api/v1/auth/register	Register a new user account	No	Public
POST	/api/v1/auth/login	Log in and receive a JWT	No	Public
POST	/api/v1/auth/logout	Invalidate the current token	Yes	All
POST	/api/v1/auth/refresh-token	Refresh an expiring JWT	Cookie only	All

6.2 Pets

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/pets	Get all pets with optional filters	No	Public
GET	/api/v1/pets/:id	Get a single pet by ID	No	Public
POST	/api/v1/pets	Create a new pet profile	Yes	Shelter Staff
PUT	/api/v1/pets/:id	Update a pet profile	Yes	Shelter Staff
DELETE	/api/v1/pets/:id	Delete a pet profile	Yes	Shelter Staff, Admin
GET	/api/v1/pets/:id/photos	Get all photos for a pet	No	Public
POST	/api/v1/pets/:id/photos	Upload a photo for a pet	Yes	Shelter Staff
POST	/api/v1/pets/:id/favorites	Add adopter's saved pets	Yes	Adopter
DELETE	/api/v1/adopters/:id/favorites	Remove a pet from favorites	Yes	Adopter
GET	/api/v1/pets/featured	Featured, available pets for the Home page	No	Public
GET	/api/v1/species	All species, alphabetical	No	Public
GET	/api/v1/breeds	Breeds, cascading from species (repeatable ?speciesID=)	No	Public

6.3 Adopters

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/adopters/me	Get an adopter profile	Yes	Adopter, Admin

Method	Endpoint	Description	Auth	Roles
PUT	/api/v1/adopters/me	Update an adopter profile	Yes	Adopter
DELETE	/api/v1/adopters/me	Delete an adopter account	Yes	Adopter, Admin
DELETE	/api/v1/adopters/me/favorites/:petId	Remove a pet from favorites	Yes	Adopter
GET	/api/v1/adopters/me/applications	Get adopter's applications	Yes	Adopter
GET	/adopters/me/favorites	Get adopter’s favorites	Yes	Adopter
GET/POST	/adopters/me/visits	Get adopter’s visits	Yes	Adopter
GET	/adopters/me/appointments	Get adopter’s pet appointments	Yes	Adopter
GET	/adopters/me/adopted-pets	Get adopter’s pets	Yes	Adopter
GET	/adopters/me/adopted-pets/:petId	Get adopted pet details	Yes	Adopter
GET	/adopters/me/adopted-pets/:petId/vaccinations	Get pet’s vaccinations	Yes	Adopter
GET/POST	/adopters/me/government-id	Get or update adopter’s goverment-id	Yes	Adopter
PATCH	/adopters/me/onboarding-step	Update adopter’s most recent onboarding step	Yes	Adopter
PATCH	/adopters/me/onboarding-complete	Update onboarding flag	Yes	Adopter

6.4 Adoption Application

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/adoption-applications	Get all applications	Yes	Shelter Staff, Admin
GET	/api/v1/adoption-applications/:id	Get a single application	Yes	Adopter, Shelter Staff
POST	/api/v1/adoption-applications	Starts a Stripe Checkout Session and returns a checkoutUrl; the actual application row is only created later, by the Stripe webhook once payment is confirmed	Yes	Adopter
PATCH	/api/v1/adoption-applications/:id/status	{ status: "Withdrawn" }, from Pending/Accepted, fee non-refundable	Yes	Adopter
GET	/adoption-applications?checkoutSessionId=	the confirmation-page polling endpoint (returns the row once the webhook has landed, or null while still pending)	Yes	Adopter

6.5 Shelters

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/shelters	Get all shelters	No	Public
GET	/api/v1/shelters/nearby	Find shelters near a lat/lng or postalCode (radius param, default 25km)	No	Public

6.6 Staff

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/staff/:id	Get a staff member profile	Yes	Admin, Shelter Staff
POST	/api/v1/staff	Create a staff account	Yes	Admin
PUT	/api/v1/staff/:id	Update a staff profile	Yes	Admin, Shelter Staff
DELETE	/api/v1/staff/:id	Delete a staff account	Yes	Admin

6.7 Appointments

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/appointments	Get all appointments	Yes	Shelter Staff, Vet
GET	/api/v1/appointments/:id	Get a single appointment	Yes	Shelter Staff, Vet, Adopter
POST	/api/v1/appointments	Create a new appointment	Yes	Vet, Shelter Staff
PATCH	/api/v1/appointments/:id	Update an appointment	Yes	Shelter Staff, Vet
DELETE	/api/v1/appointments/:id	Cancel an appointment	Yes	Shelter Staff, Adopter

6.8 Vaccinations

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/pets/:id/vaccinations	Get vaccinations history for a pet	Yes	Vet, Shelter Staff
POST	/api/v1/pets/:id/vaccinations	Record a vaccination	Yes	Vet

Method	Endpoint	Description	Auth	Roles
PATCH	/api/v1/pets/:id/vaccinations/:recordId	Record a vaccination for a pet	Yes	Vet

administeredBy (vet) and administeredAt (shelter) are fields on the vaccination record itself, set in the request body — not separate path segments — since a vaccination isn't owned by one specific appointment in the data model.

Vaccinations nest under pets, not appointments: VACCINATION_RECORD has no appointmentID — it links to petID directly, with administeredBy (vet) and administeredAt (shelter) as independent fields. A pet can be vaccinated outside any tracked appointment, so /pets/:id/ vaccinations reflects the real ownership; nesting under /appointments/:id would imply a foreign key that doesn't exist.

6.9 Tasks & Assignments

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/tasks	Get all tasks	Yes	Shelter Staff, Admin
POST	/api/v1/tasks	Create a new task	Yes	Shelter Staff
GET	/api/v1/tasks/:id	Get a single task	Yes	Shelter Staff, Volunteer
POST	/api/v1/tasks/:id/assign	Assign a task to a staff member	Yes	Shelter Staff
PATCH	/api/v1/tasks/:id/assignments/:staffId	Update task status	Yes	Shelter Staff, Volunteer

6.10 Events

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/events	Get all events	No	Public
GET	/api/v1/events/:id	Get a single event	No	Public
POST	/api/v1/events	Create a new event	Yes	Shelter Staff
PUT	/api/v1/events/:id	Update an event	Yes	Shelter Staff

Method	Endpoint	Description	Auth	Roles
DELETE	/api/v1/events/:id	Delete an event	Yes	Shelter Staff, Admin

6.11 Donors & Donations

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/donors/:id	Get a donor profile	Yes	Donor, Admin
POST	/api/v1/donors	Create a donor account	No	Public
PUT	/api/v1/donors/:id	Update a donor profile	Yes	Donor
GET	/api/v1/donations	Get all donations	Yes	Admin, Shelter Staff
POST	/api/v1/donations	Make a donation (triggers Stripe)	Yes	Donor
GET	/api/v1/donations/:id	Get a single donation record	Yes	Donor, Admin

6.12 Shelter Transfers

Method	Endpoint	Description	Auth	Roles
GET	/api/v1/transfers	Get all transfer requests	Yes	Shelter Staff, Admin
GET	/api/v1/transfers/:id	Get a single transfer request	Yes	Shelter Staff, Admin
POST	/api/v1/transfers	Initiate a new transfer request	Yes	Shelter Staff
PATCH	/api/v1/transfers/:id/status	Approve or decline a transfer	Yes	Shelter Staff, Admin

6.13 Visits

Method	Endpoint	Description	Auth	Roles
POST	/api/v1/visits	Add a visit	Yes	Adopter, Shelter Staff
PATCH	/api/v1/visits/:id	Update a visit	Yes	Adopter, Shelter Staff

7. Design Decisions & Rationale

This section documents key design choices made during API planning and the reasoning behind them.

1. **REST over GraphQL**: REST is simpler to implement, easier to test with Postman, and aligns with the project's existing Express/Node.js stack. GraphQL's flexibility is not required for this project's scope.
2. **URL-based versioning (`/api/v1/`) over header-based versioning**: URL versioning is more explicit, easier to test, and more readable in logs and Swagger documentation.
3. **One-level nesting only:** Deep nesting (e.g. `/shelters/:id/staff/:staffId/tasks`) becomes hard to maintain. Keeping nesting to one level keeps URLs readable and routing logic simple.
4. **Shelter transfers as a dedicated resource**: Transfers involve two shelters, a status lifecycle, and future audit logging. Treating them as a first-class resource rather than a pet update keeps the model clean.
5. **`PATCH` for status updates** — Using `PATCH` (partial update) for status changes (e.g. application approval, transfer approval) is semantically correct and avoids requiring the full resource body in the request.
6. **Stripe delegated to POST /donations** — Payment processing logic is delegated to Stripe on the backend. The API endpoint triggers the Stripe flow rather than handling card data directly, keeping PCI compliance concerns out of scope.